React Native Take-Home: Live Portfolio

Overview

Build a small stock portfolio app in React Native that stays correct and smooth while prices tick several times a second.

You have 3 calendar days to submit, so fit it around your schedule. If you run out of time, stop, and tell us in the README what you would do next. A finished core beats a half-built everything.

We care most about three things:

- **Performance:** a list of live prices that does not re-render everything on every tick.
- **Correctness:** money values that are never wrong, and never faked when data is missing.
- **Structure:** code a teammate could pick up and extend.

Use any tools you like, including AI assistants. In the next round we will ask you to walk through your code and extend it live, so make sure you understand every part of it.

What to build

The app has three screens plus a buy/sell sheet, all driven by the same live prices.

1. Portfolio (home)

- Summary card: current value, total invested, total P&L, and today's P&L, each in ₹ and %.
- Holdings list: symbol, quantity, average price, last price, and P&L in ₹ and %.
- Green for gain, red for loss. Each row updates live.
- Sort by name, value, or P&L. Filter by symbol as you type.
- Pull to refresh re-fetches prices from the API.

2. Watchlist

- Symbol, last price, and day change in ₹ and %.
- Add a symbol from the list provided below, and remove with a swipe.
- The watchlist survives an app restart.

3. Stock detail

- Opens from any row on Portfolio or Watchlist.
- Shows last price, day change, open, high, low, previous close, and volume, all live.

- A small line chart of the ticks received since the screen opened.
- Your position in this stock, if you hold it.

4. Buy/sell sheet

- Opens from Stock detail. Choose buy or sell, enter a quantity, and see the estimated value at the live price.
- Validate the input: whole numbers above zero, and no selling more than you hold.
- On confirm, fill at the last price and update holdings straight away. Average price changes on a buy. Holdings and Summary must stay consistent.
- Orders are local only. No real order is placed.

Every screen also needs:

- Loading, empty, and error states, with retry on error.
- A visible "prices paused" indicator when ticks stop.
- Light and dark mode, following the system setting.
- Indian number formatting, for example ₹12,34,567.89.

Data and price feed

Real prices come from the Liquide OHLC API once at start and on refresh; you simulate live ticks on top of them.

OHLC API. No login is needed.

```
curl -X POST https://api.v2.liquide.life/api/markets/ohlc \
  -H 'content-type: application/json' \
  -d '{"symbols":["RELIANCE"]}'
```


```
{
  "status": "success",
  "code": 200,
  "count": 1,
  "message": {
    "data": [
      {
        "symbol": "RELIANCE",
        "isin": "INE002A01018",
        "ltp": 1164.9,
        "open": 1170.4,
        "prevClose": 1167.4,
        "high": 1174.6,
        "low": 1152.5,
```

```
        "volume": 14753650,
        "timeStamp": "2026-10-30T15:59:52",
        "scripId": "62584d19d8ddbbdfbe1ccfcf",
        "navDate": null,
        "tsn": 13012
      }
    ]
  }
}
```

Things to know about this API:

- The request returns HTTP 201 on success. The list sits at `message.data`.
- A symbol it does not recognise is left out of the response, with no error.
- Outside market hours, prices are the last close. That is expected, and your simulator keeps the app moving.
- This is a shared test server. Fetch once per load, and do not poll it.

Symbols. Use these 25 for the app and for the watchlist picker:

RELIANCE, TCS, HDFCBANK, INFY, ICICIBANK, SBIN, BHARTIARTL, ITC, LT, HINDUNILVR, KOTAKBANK, AXISBANK, BAJFINANCE, MARUTI, SUNPHARMA, TITAN, ASIANPAINT, ULTRACEMCO, WIPRO, NTPC, POWERGRID, ONGC, TATASTEEL, ADANIENT, HCLTECH

Starting holdings. Bundle these with the app. `OLDCO` is deliberate: the API will not return it.

```
symbol,qty,avgPrice
RELIANCE,25,1319.59
TCS,40,2589.71
HDFCBANK,10,755.86
INFY,10,1114.7
ICICIBANK,5,1566.04
SBIN,15,800.62
BHARTIARTL,40,1890.8
ITC,15,225.74
LT,40,3250.32
HINDUNILVR,100,1605.15
KOTAKBANK,15,474.57
AXISBANK,100,1378.55
BAJFINANCE,100,951.52
MARUTI,5,13017.25
SUNPHARMA,5,1784.28
TITAN,12,4061.93
ASIANPAINT,12,2316.35
```

```
ULTRACEMCO,100,10181.27
WIPRO,12,169.36
NTPC,100,326
OLDCO,30,412.5
```

Tick simulator. Write this as a separate module that behaves like a socket client, with `connect()`, `disconnect()`, and `onTick(handler)`. Your screens should not be able to tell it is fake.

- Emit about 20 ticks per second across all symbols.
- Each tick moves one symbol's `ltp` by up to ±0.2% and may also change `high`, `low`, and `volume`.
- A tick is a partial update: `{ symbol, seq, ltp?, high?, low?, volume? }`. Only changed fields are present.
- `seq` increases per symbol. About 2% of ticks arrive late, out of order, and must be ignored.
- Roughly once a minute the feed goes silent for 5 seconds, then resumes. This drives the "prices paused" indicator.

Missing data. If a price is unavailable, show `--`. Never show 0 or a guessed value, and never count an unpriced holding as a zero in the totals. Decide how Summary should present this, and explain the choice in your README.

Technical requirements

These are firm; everything else is your call.

- React Native with TypeScript in strict mode. Expo is fine, and bare React Native is too.
- It must run on Android or iOS. Tell us which one you tested.
- Use any state, navigation, and chart libraries you like. Be ready to explain why you chose them.
- You will encounter re-rendering issues, and we want to see how you address them. We will evaluate your solution using the React DevTools Profiler.
- The list must stay smooth while scrolling at 20 ticks per second.
- Keep P&L and price calculations in plain functions, outside components, with unit tests.
- Handle money rounding deliberately. Summary totals must match the sum of the rows as displayed, or the README must say why not.
- Use a theme for light and dark mode.

Stretch goals (optional)

Do these only if the core is finished; they earn credit but never make up for gaps in it.

- Price cell flash on change, running on the UI thread.
- Reconnect with backoff, plus a catch-up REST fetch when the feed returns.
- Limit orders that fill when the simulated price crosses the limit.
- Orders history screen, persisted across restarts.
- Simulator controls in a dev menu: tick rate and force disconnect.

Submission

Send a Git repository link within 3 calendar days of receiving this brief.

Keep your commit history; please do not squash it into one commit. Include a 1–2 minute screen recording of the app running on a device or emulator.

Your README should cover:

- How to install and run the app.
- Your state and re-render approach, and why you chose it.
- How Summary handles a holding with no price.
- What you skipped, and what you would do next with another day.
- Roughly how long you spent.

How we evaluate

Performance and correctness carry the most weight, because a trading app fails its users first on those two.

Area	Weight	What we look for
Performance	30%	Rows update in isolation, scrolling stays smooth under ticks, no needless re-renders
Correctness	25%	P&L and averages are right, late ticks are ignored, `--` instead of fake values, totals reconcile
Architecture and state	20%	Clear separation of feed, state, and UI; simulator swappable for a real socket
UI and edge cases	15%	Loading, empty, error, and paused states; both themes; number formatting

Area	Weight	What we look for
TypeScript and tests	10%	Strict types without `any`, meaningful tests on calculations

We do not reward extra features over a solid core. In the next round we will review your code together and ask you to make a small live change to it.